Cifrado con AES y ChaCha20 desde cero

Guía práctica paso a paso para cifrar archivos, atacarlos y entender por qué el modo de operación importa tanto como el algoritmo.

Ana Arias, Allyson Sequeira y Johana Solano

2026

1.¿Qué vamos a hacer?
Al terminar esta guía habrás ejecutado una herramienta real de cifrado y habrás visto con tus propios ojos tres cosas que normalmente solo se leen en libros:

Qué harás	Qué verás
Experimento 1 El “pingüino ECB”	Cifrarás una imagen con AES-ECB y con AES-GCM. En ECB todavía se ve la silueta de la imagen; en GCM solo ves ruido.
Experimento 2 Manipulación	Cambiarás un solo byte de un archivo cifrado. Con GCM el descifrado falla y te avisa; con CTR “funciona”… pero con datos alterados.
Experimento 3 Rendimiento	Medirás qué tan rápidos son AES-128-GCM, AES-256-GCM y ChaCha20-Poly1305 en tu máquina y verás un gráfico.

2.Preparar tu máquina virtual
Necesitas una VM con Ubuntu 22.04/24.04 (o Debian 12), conexión a internet, ~500 MB libres y un usuario con permisos sudo. En macOS o en un Linux normal también funciona; solo cambia la instalación de paquetes.

3.Obtener el proyecto
Elige una de estas dos opciones.

Comando	Qué hace
scp Algoritmos-AES-CHACHA20.zip usuario@IP_DE_LA_VM:~/	Opción A. Copia el ZIP desde tu computadora a la VM (cambia usuario e IP_DE_LA_VM). Si tu VM tiene carpetas compartidas, basta con arrastrar el archivo.
unzip Algoritmos-AES-CHACHA20.zip	Descomprime el ZIP; crea la carpeta Algoritmos-AES-CHACHA20.
git clone https://github.com/anamcampos/Proyecto-4-Cifrado-sim-trico-AES-y-ChaCha20.git	Opción B. Descarga el proyecto desde GitHub (cambia TU_USUARIO por el usuario del repositorio).
cd Algoritmos-AES-CHACHA20	Entra a la carpeta del proyecto. Todos los comandos de esta guía se ejecutan desde aquí.

4.Instalar lo necesario

Comando	Qué hace
sudo apt update	Actualiza la lista de paquetes disponibles (no instala nada todavía). Pedirá tu contraseña.
sudo apt install -y python3 python3-venv python3-pip unzip git openssl	Instala Python 3, el creador de entornos virtuales, pip, unzip, git y openssl. -y responde “sí” a las preguntas.
python3 -m venv .venv	Crea un entorno virtual: una carpeta oculta .venv con su propio Python, para no mezclar librerías con las del sistema.
source .venv/bin/activate	Activa ese entorno. Verás (.venv) al inicio de la línea. Repite este comando cada vez que abras una terminal nueva.
pip install -r requirements.txt	Instala las librerías que lista requirements.txt: cryptography (el cifrado), Pillow (imágenes) y matplotlib (gráficos).
python3 cifrador.py --version	Comprueba que la herramienta arranca.
python3 -c "import cryptography, PIL, matplotlib; print('Todo OK')"	Intenta importar las tres librerías; si imprime “Todo OK”, la instalación está completa.

5.Archivo cifrador.py

La herramienta tiene tres subcomandos: generar-llave, cifrar y descifrar. Puedes ver la ayuda en cualquier momento con:

Comando	Qué hace
python3 cifrador.py --help	Muestra los subcomandos disponibles.
Comando	Qué hace

python3 cifrador.py cifrar --help	Muestra las opciones de un subcomando concreto (cambia cifrar por otro).

Opción	Qué significa
--modo, -m	ecb, gcm, chacha20 (o chacha20-poly1305) y ctr (solo didáctico). Obligatorio al cifrar. Al descifrar es opcional: el modo se lee de la cabecera del archivo.
--bits	Solo en generar-llave: tamaño de la llave AES, 128, 192 o 256 (por defecto 256). ChaCha20 siempre usa 256.
--llave, -k	Archivo donde se guarda (generar-llave) o de donde se lee (cifrar/descifrar) la llave.
--entrada, -i	Archivo que quieres cifrar o descifrar.
--salida, -o	Archivo resultante. Si no lo das: al cifrar se añade .enc; al descifrar se quita .enc.
--forzar	Permite sobrescribir un archivo que ya existe. Sin esta opción la herramienta se niega, para protegerte.

6.Cómo es un archivo cifrado por dentro

Cada archivo cifrado empieza con una pequeña cabecera para que la herramienta sepa cómo descifrarlo:

Parte	Tamaño	Para qué sirve
“CIFR”	4 bytes	Firma que identifica archivos creados por cifrador.py.
Versión	1 byte	Versión del formato (hoy: 1).
Modo	1 byte	1 = ECB, 2 = GCM, 3 = ChaCha20-Poly1305, 4 = CTR.
Nonce	12 bytes (GCM/ChaCha20), 16 (CTR), 0 (ECB)	Número de un solo uso, generado al azar en cada cifrado.

Texto cifrado	Resto del archivo	Tus datos cifrados. En GCM y ChaCha20 los últimos 16 bytes son la etiqueta de autenticación.

Detalle de seguridad

En GCM y ChaCha20-Poly1305 la cabecera de 6 bytes también queda autenticada: si alguien cambia el byte del modo, por ejemplo, el descifrado se rechaza igualmente.

7.Códigos de salida

Código	Significado
0	Todo salió bien.
1	Error de uso: archivo o llave inexistente, llave de tamaño incorrecto, el modo pedido no coincide con el del archivo, etc.
2	ARCHIVO MANIPULADO. Falló la verificación de autenticidad. No se escribe ningún archivo de salida.

Ahora bien, antes de comenzar con los experimentos, vamos a hacer unas pequeñas prácticas para comprender mejor los cifrados.

Práctica guiada con cifrador.py

Antes de los experimentos, practica con la herramienta en seis pasos cortos. Trabaja en una carpeta de pruebas para no mezclar archivos:

Comando	Qué hace
mkdir -p ~/practica && cp cifrador.py ~/practica/ && cd ~/practica	Crea la carpeta practica en tu casa, copia ahí la herramienta y entra a ella. (&& significa “si lo anterior salió bien, continúa”).

Práctica 1 · Crear un archivo y una llave

Comando	Qué hace
echo "Mensaje muy secreto: la nota final es 100" > secreto.txt	Crea el archivo secreto.txt con ese texto (42 bytes). El símbolo > redirige el texto al archivo.
python3 cifrador.py generar-llave --modo gcm --bits 256 --llave mi.key	Genera una llave aleatoria de 256 bits y la guarda en mi.key, con permisos para que solo tú puedas leerla.

cat mi.key	Muestra la llave: 64 caracteres hexadecimales (= 32 bytes = 256 bits).

```
[OK] Llave de 256 bits guardada en 'mi.key' (permisos 600).
      Guárdala APARTE de los archivos cifrados y no la subas a GitHub.
26a71f787f0040ce0fd5ebc17fc10af928a0728a40b19bbd2268d4040cbfa5e4      <-
la tuya será distinta
```

> Importante
>
> Si pierdes la llave, no hay forma de recuperar los archivos cifrados. Si intentas crear una llave sobre otra existente, la herramienta se negará: *[ERROR] 'mi.key' ya existe… (¡los archivos cifrados con la llave anterior quedarían ilegibles!)*.

Práctica 2 · Cifrar con AES-GCM y mirar el resultado

Comando	Qué hace
python3 cifrador.py cifrar --modo gcm --llave mi.key -i secreto.txt -o secreto.enc	Cifra secreto.txt con AES-GCM usando mi.key y guarda el resultado en secreto.enc.
ls -l secreto.txt secreto.enc	Lista los archivos con su tamaño. El cifrado pesa más: 42 B + 6 de cabecera + 12 de nonce + 16 de etiqueta = 76 bytes.
head -c 64 secreto.enc | od -A d -t x1z	head -c 64 toma los primeros 64 bytes; od los muestra en hexadecimal (-t x1) con su posición decimal (-A d) y, a la derecha, como texto (z).

```
[OK] 'secreto.txt' cifrado con AES-GCM (cifra y autentica) -> 'secreto.enc' 0000000
43 49 46 52 01 02 c5 37 50 cf f6 a7 cc e2 4e e6  >CIFR...7P.....N.<
0000016 c3 22 15 4a f2 f5 1e 25 4c d0 0a 02 69 13 80 8c  >.".J...%L...i...<
0000032 d0 63 b2 b8 72 95 3a 57 bb 5e 4d 71 06 b4 19 85  >.c..r.:W.^Mq....<
0000048 6d 49 ca 8f 1d 1e 04 f1 29 81 3d 7d 4a b6 df 38  >mI......).=}J..8<
```

Cómo leer esos bytes: 43 49 46 52 son las letras “CIFR”; 01 es la versión; 02 es el modo (2 = GCM); los siguientes 12 bytes (c5 37 … c3 22) son el nonce; el resto es tu mensaje cifrado y la etiqueta. Fíjate en que no se entiende nada del texto original.

Práctica 3 · Descifrar

Comando	Qué hace

python3 cifrador.py descifrar --llave mi.key -i secreto.enc -o recuperado.txt	Descifra secreto.enc. No hace falta decir el modo: se lee de la cabecera. Verifica la etiqueta antes de entregar nada.
cat recuperado.txt	Muestra el texto recuperado; debe ser idéntico al original.

```
[OK] 'secreto.enc' descifrado con AES-GCM (cifra y autentica) -> 'recuperado.txt'      Autenticidad verificada: el contenido no fue alterado.
Mensaje muy secreto: la nota final es 100
```

Práctica con ChaCha20-Poly1305 y una llave equivocada

Comando	Qué hace
python3 cifrador.py generar-llave -m chacha20 -k chacha.key	Crea una llave nueva de 256 bits para ChaCha20-Poly1305 (-m y -k son abreviaturas de --modo y --llave).
python3 cifrador.py cifrar -m chacha20 -k chacha.key -i secreto.txt -o secreto_cha.enc	Cifra con ChaCha20-Poly1305.
python3 cifrador.py descifrar -k chacha.key -i secreto_cha.enc -o rec_cha.txt	Descifra con la llave correcta: funciona.
python3 cifrador.py descifrar -k mi.key -i secreto_cha.enc -o rec_cha2.txt	Intenta descifrar con la llave de GCM (equivocada): falla.

```
[ALERTA] ARCHIVO MANIPULADO O CORRUPTO          Falló la verificación de autenticidad: el archivo fue MODIFICADO o la llave no es la correcta.
        No se escribió ningún archivo de salida.
```

Observa que el mensaje menciona dos causas posibles: archivo modificado o llave incorrecta.
Matemáticamente ambas situaciones son indistinguibles: en los dos casos la etiqueta no coincide.

Práctica Alterar un byte a mano

Ahora actuarás como el atacante: cambiarás un byte del archivo cifrado con GCM.

Comando	Qué hace
cp secreto.enc secreto_ALTERADO.enc	Hace una copia para no estropear el original.

printf '\000' | dd of=secreto_ALTERADO.enc bs=1 seek=30 count=1 conv=notrunc	Escribe un solo byte con valor 0 en la posición 30 de la copia. bs=1 count=1: un byte; seek=30: salta 30 bytes; conv=notrunc: no recorta el resto del archivo.
cmp -l secreto.enc secreto_ALTERADO.enc	Compara ambos archivos y lista las diferencias (posición y valores en octal). Debe aparecer una sola línea: ese es tu byte cambiado.
Comando	**Qué hace**
python3 cifrador.py descifrar -k mi.key -i secreto_ALTERADO.enc -o x.txt	Intenta descifrar el archivo alterado.
echo $?	Imprime el código de salida del comando anterior: debe ser 2.
ls x.txt	Comprueba que no existe: la herramienta no entregó datos no verificados.
31 200 0 [ALERTA] ARCHIVO MANIPULADO O CORRUPTO Falló la verificación de autenticidad: el archivo fue MODIFICADO o la llave no es la correcta. No se escribió ningún archivo de salida. 2 ls: cannot access 'x.txt': No such file or directory	

Por si cmp no muestra nada

Hay una probabilidad de 1 entre 256 de que el byte original ya valiera 0 y no cambie nada. Si cmp no imprime líneas, repite el printf con otra posición (por ejemplo seek=31).

Práctica Ver el patrón de ECB

Comando	Qué hace
printf 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA' > a64.txt	Crea un archivo con 64 letras “A” (4 bloques idénticos de 16 bytes).
python3 cifrador.py generar-llave -m ecb -k ecb.key	Crea una llave para el modo ECB.

python3 cifrador.py cifrar -m ecb -k ecb.key -i a64.txt -o a64_ecb.enc	Cifra con ECB (la herramienta te avisará que no autentica).
tail -c +7 a64_ecb.enc | od -v -A d -t x1	tail -c +7 salta la cabecera de 6 bytes; od -v muestra todos los bytes sin abreviar. Verás cuatro líneas idénticas (sección 3.2).
python3 cifrador.py cifrar -m gcm -k mi.key -i a64.txt -o a64_gcm.enc	Cifra los mismos datos con GCM.
tail -c +19 a64_gcm.enc | od -v -A d -t x1	tail -c +19 salta cabecera + nonce (18 bytes). Ahora ninguna línea se repite.

Ahora haremos los experimentos:

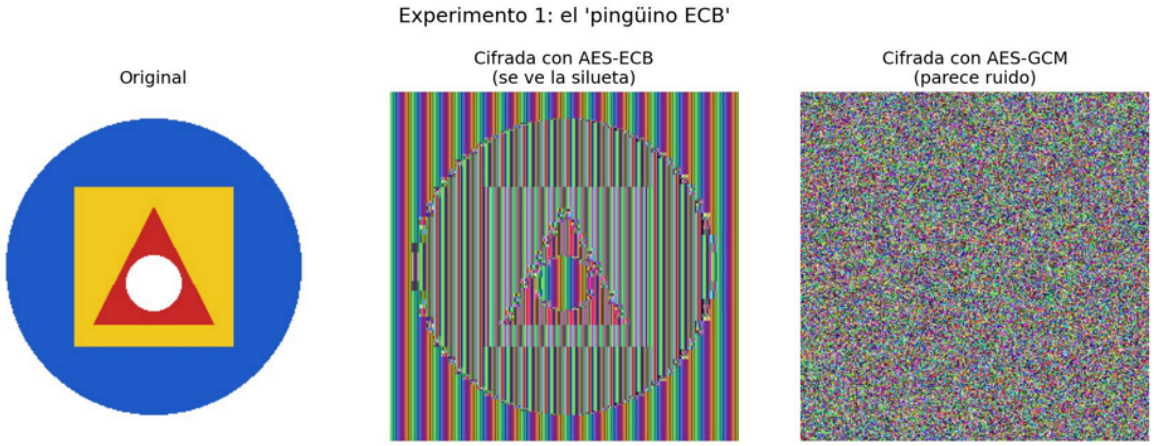
Experimento 1 - El pingüino ECB

Objetivo: ver que ECB deja filtrar la forma de una imagen aunque esté “cifrada”.
Archivo: experimento1_pinguino_ecb.py

Comando	Qué hace
cd ~/Algoritmos-AES-CHACHA20	Vuelve a la carpeta del proyecto (ajusta la ruta si la descomprimiste en otro lugar).
source .venv/bin/activate	Activa el entorno virtual (si tu terminal no muestra (.venv)).
python3 experimento1_pinguino_ecb.py	Dibuja un logo de colores planos, cifra sus píxeles con ECB y con GCM y guarda las imágenes.
Comando	**Qué hace**
python3 experimento1_pinguino_ecb.py --imagen mi_logo.png	*(Opcional)* Usa tu propia imagen; funciona mejor con colores planos.
ls resultados/exp1/	Lista las imágenes generadas.

```
Imágenes guardadas en
/home/usuario/Algoritmos-AES-CHACHA20/resultados/exp1
Bloques de 16 bytes en la imagen original : 94 únicos de 12288 bloques
Bloques de 16 bytes cifrados con ECB      : 94 únicos de 12288 bloques
Bloques de 16 bytes cifrados con GCM      : 12288 únicos de 12288 bloques ->
En ECB casi todos los bloques se repiten (hay patrón); en GCM casi todos son
distintos.
```

Cómo funciona el script: toma los bytes crudos de los píxeles (R, G, B, R, G, B…), los cifra con AES-ECB y con AES-GCM, y vuelve a armar una imagen con los bytes cifrados. Así se ve directamente qué “filtra” cada modo.



Cómo interpretarlo

- En ECB, zonas del mismo color producen los mismos bloques cifrados, así que el contorno de cada figura sigue siendo visible. Las rayas verticales aparecen porque un bloque son 16 bytes y un píxel son 3, así que el patrón se repite cada 3 bloques (16 píxeles).
- La clave está en el contador de “bloques únicos”: el original y su versión ECB tienen exactamente el mismo número (94). ECB solo “cambia los colores” de cada bloque, no destruye la estructura. GCM produce 12 288 bloques distintos de 12 288: ningún patrón sobrevive.
- Lo que debes entregar: la imagen comparacion_ecb_gcm.png y una frase explicando por qué ECB filtra la silueta.

Experimento 2 - Manipulación de archivos cifrados

Objetivo: comprobar que los modos con autenticación detectan un byte cambiado y los modos sin autenticación no. Archivo: experimento2_manipulacion.py. El script usa cifrador.py real, así que verás exactamente lo que verías a mano.

Comando	Qué hace
python3 experimento2_manipulacion.py	Para cada modo (gcm, chacha20, ctr, ecb): genera una llave, cifra un mensaje, descifra el original, cambia un byte y vuelve a descifrar.
cat resultados/exp2/resumen.txt	Muestra el registro completo guardado en un archivo (útil para pegarlo en tu informe).

ls resultados/exp2/	Lista los archivos cifrados y los alterados (*_ALTERADO.enc).

El mensaje de prueba es: ORDEN DE PAGO: transferir 100 colones a Ana. Cuenta destino:
123-456. El “atacante” cambia un solo byte. En GCM y ChaCha20-Poly1305 obtendrás:

```
[ALERTA] ARCHIVO MANIPULADO O CORRUPTO          Falló la verificación de autenticidad: el archivo fue MODIFICADO o la llave no es la correcta.
        No se escribió ningún archivo de salida.
(código de salida: 2)
```

Y en CTR (sin autenticación) el descifrado “funciona”, pero:

```
  Texto original : ORDEN DE PAGO: transferir 100 colones a Ana. Cuenta destino: 123-456.
  Texto recibido : ORDEN DE PAGO: transferir 900 colones a Ana. Cuenta destino: 123-456.
```

Al final el script imprime esta tabla resumen:

```
Modo          ¿Detectó la manipulación?   Qué pasó
gcm           SI (rechazó el archivo)     
chacha20      SI (rechazó el archivo)     
ctr           NO (aceptó datos alterados) el texto dice 900 colones en vez de 100
ecb           NO (aceptó datos alterados) un bloque de 16 bytes quedó como basura
```

Cómo interpretarlo

- GCM y ChaCha20-Poly1305: la etiqueta de autenticación no coincide, así que el descifrado se rechaza y no se escribe ningún archivo. Lo único que cambió fue un byte, y se detectó.
- CTR: el texto cifrado es *texto* ⊕ *flujo secreto*. Si el atacante cambia un byte del cifrado, cambia exactamente ese byte del texto claro al descifrar. Sabiendo dónde está el “1” de “100”, basta calcular el XOR entre ‘1’ y ‘9’ para convertirlo en “900”. El atacante no necesita la llave y el receptor no recibe ningún aviso.
- ECB: no hay aviso, y el bloque de 16 bytes afectado se convierte en basura aleatoria.
- Lección: cifrar protege la *confidencialidad*, pero no la *integridad*. Para ambas usa GCM o ChaCha20-Poly1305.

Experimento 3 - Rendimiento

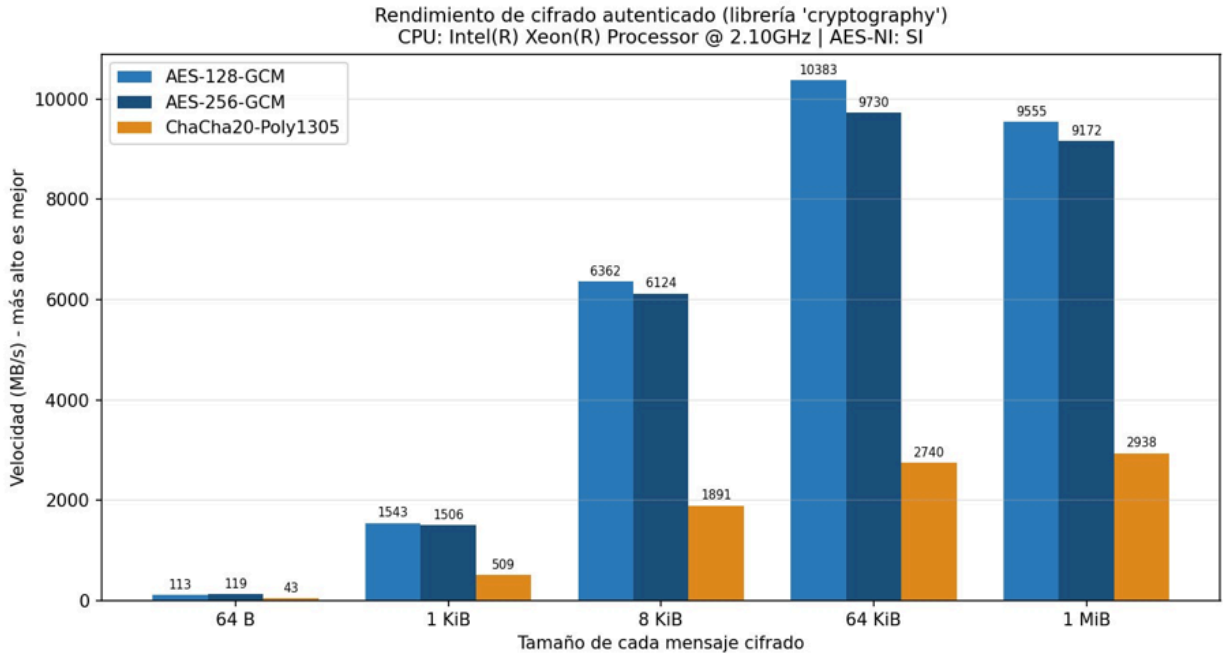
Objetivo: comparar la velocidad de AES-128-GCM, AES-256-GCM y ChaCha20-Poly1305. Archivo: experimento3_rendimiento.py

Comando	Qué hace
grep -o aes /proc/cpuinfo | head -1	Comprueba si tu CPU tiene AES-NI (ver sección 4.3). Anota el resultado.
python3 experimento3_rendimiento.py --segundos 3	Mide los tres algoritmos con 5 tamaños de mensaje (64 B a 1 MiB), unos 3 s por combinación (~45 s en total), y crea el gráfico y un CSV.
Comando (cópialo tal cual)	**Qué hace**
python3 experimento3_rendimiento.py --openssl	Hace lo mismo y además ejecuta openssl speed para comparar con la herramienta de referencia.
openssl speed -evp aes-256-gcm	*(Manual)* Benchmark oficial de OpenSSL para AES-256-GCM. -evp usa la interfaz moderna, que aprovecha AES-NI.
openssl speed -evp chacha20-poly1305	*(Manual)* Lo mismo para ChaCha20-Poly1305.
cat resultados/exp3/rendimiento.csv	Muestra la tabla de resultados en formato CSV (abre en Excel o LibreOffice).

Cómo leer la salida de openssl speed. Los números están en miles de bytes por segundo (k): 13 569 000k ≈ 13,6 GB/s. Cada columna es un tamaño de bloque (16, 64, 256, 1024, 8192 y 16384 bytes). Con bloques muy pequeños casi todo el tiempo se gasta en “arrancar” cada operación, por eso las primeras columnas son mucho más lentas.

Resultado de ejemplo (equipo donde se preparó este material: Intel Xeon 2,10 GHz con AES-NI), en MB/s:

Algoritmo	64 B	1 KiB	8 KiB	64 KiB	1 MiB
AES-128-GCM	113	1 543	6 362	10 383	9 555
AES-256-GCM	119	1 506	6 124	9 730	9 172
ChaCha20-Poly1305	43	509	1 891	2 740	2 938



Cómo interpretarlo

- Con AES-NI, AES-GCM es varias veces más rápido que ChaCha20-Poly1305 (en el ejemplo, ~3 veces con mensajes grandes). AES-256 es solo ~4 % más lento que AES-128: pasar a una llave más larga cuesta muy poco.
- Sin AES-NI (CPU antigua o VM que no la expone) AES debe hacerse por software y suele perder frente a ChaCha20, que fue diseñado para ser rápido en cualquier procesador. Por eso ChaCha20-Poly1305 es popular en celulares.
- Tus resultados serán distintos: dependen de tu CPU y de tu VM. Eso es normal y es lo que se espera. Entrega tu gráfico y comenta si tu CPU tiene AES-NI.
- Los mensajes diminutos (64 B) son lentos en Python porque domina el costo de cada llamada, no elcifrado en sí.

En caso que algo salga mal:

Mensaje o síntoma	Qué hacer
ModuleNotFoundError: No module named 'cryptography' (o PIL / matplotlib)	No activaste el entorno virtual o no instalaste las librerías. Ejecuta source .venv/bin/activate y luego pip install -r requirements.txt.
error: externally-managed-environment	Estás instalando con pip fuera del entorno virtual. Crea y activa el entorno (sección 4.2) y vuelve a instalar.

python3: command not found	Instala Python: sudo apt install -y python3 python3-venv python3-pip.
[ERROR] No encuentro el archivo de llave	Aún no la creaste o estás en otra carpeta. Ejecuta generar-llave y revisa con ls que el archivo exista.
[ERROR] … ya existe. Usa --forzar	El archivo de destino ya existe. Cambia el nombre con -o o agrega --forzar. Para llaves, recuerda que reemplazarlas deja ilegibles los archivos antiguos.
[ERROR] Pediste --modo X pero el archivo dice…	El modo que indicaste no coincide con el de la cabecera. Quita --modo al descifrar o usa el correcto.
[ALERTA] ARCHIVO MANIPULADO sin haber tocado nada	Casi seguro es una llave equivocada (por ejemplo, la de otro archivo). Comprueba que usas la llave con la que cifraste.
[ERROR] Relleno inválido (solo ECB)	Llave equivocada o archivo dañado. ECB no autentica, por lo que no puede decirte cuál de las dos.
El experimento 3 no muestra “aes”	Tu VM no expone AES-NI (ver 4.3). No es un error de la práctica.
[AVISO] No encontré 'openssl'	Instálalo con sudo apt install -y openssl y repite con --openssl.